Recovering Code Performance After Persona Fine-Tuning: A Single-Run Case Study on Qwen2.5-3B-Instruct

Riddhi Bhagwat Essa Jan Daniel Schwartz

CS 2881R, Assignment 1

github.com/essacheez/Posttraining-coding-rocky

Abstract

We fine-tune Qwen2.5-3B-Instruct to answer in the voice of Rocky from *Project Hail Mary* while retaining Python code generation. Code accuracy is reported as pass@1, the fraction of problems solved by the model's first generated program. Supervised fine-tuning (SFT) changes surface style but reduces HumanEval and HumanEval+ pass@1 by 20.7 and 23.2 percentage points. An initial reinforcement learning from AI feedback (RLAIF) run exploits an underspecified style reward; a revised objective separates task, style, format, and anti-catchphrase rewards. A final RLVR stage trained on 700 problems ordered by estimated difficulty recovers 11.6 points on HumanEval, 10.9 on HumanEval+, and 2.2 on MBPP in the training-trajectory evaluation. A separate final-model rerun obtains 0.683, 0.628, and 0.584, respectively, plus 0.149 on LiveCodeBench. The completed five-dimension persona judge scores the final model 18.5/25, including 3.19/5 for human plausibility. Each training stage was run once, so these results do not measure variation across repeated training runs. The evidence shows substantial recovery of the measured coding loss while retaining recognizable persona features, but not the absence of a general capability-persona trade-off.

1 Introduction

Post-training can install a desired behavior while degrading capabilities that the training data rarely exercises. Persona tuning makes this failure easy to observe: language style and task content share the same output channel, so examples that reward stylized prose can implicitly discourage code. We test this interaction using Qwen2.5-3B-Instruct [7], a low-rank adaptation (LoRA) pipeline [3], and a target persona with conspicuous grammatical markers.

The project asks three empirical questions. First, how much coding ability is lost when the persona is installed by SFT? Second, can an AI judge improve the persona without being gamed? Third, can execution-based reinforcement learning recover coding ability without undoing the persona? We use SFT, RLAIF [4], and group relative policy optimization (GRPO) [6] in sequence.

Our contributions are primarily diagnostic:

- We document a data-composition failure: only 21 of 1,000 SFT targets contain a fenced code block.
- We retain a failed RLAIF checkpoint as a control and show that a holistic persona score can miss obvious reward gaming.
- We compare earlier MBPP-based RLVR ablations with a final RLVR run trained on 700 problems selected from a 1,700-problem pool and ordered by estimated difficulty.
- We report the completed coding, verifier, stylometric, and five-dimension persona evaluations, including disagreements between metrics and reruns.

2 Experimental Setting

2.1 Model and training sequence

All stages adapt Qwen2.5-3B-Instruct using rank-32 LoRA modules on the model's projection layers. The deployed sequence is:

1. **SFT:** 1,200 rewritten conversations are divided into 1,000 development examples and 200 held-out persona prompts. The trainer splits the 1,000 examples 95/5, using 950 for optimization and 50 for internal validation. Training uses three epochs and response-only loss.
2. **RLAIF:** the builder assembles 600 rows: 500 assistant prompts, 60 narrative prompts drawn from the SFT training file, and 40 MBPP validation problems. A 95/5 split leaves 570 prompts for training and 30 unused holdout prompts. Eight generations are sampled per prompt and optimized with GRPO. GPT-5.4-nano supplies task and style rewards, while deterministic rules score formatting and catchphrase frequency.
3. **RLVR:** all 700 selected code problems are used for training, with no internal split. They are ordered by estimated difficulty; the reported model is the saved step-174 checkpoint from a run on two L40S GPUs.

SFT and RLAIF use the default Qwen chat template and no persona system prompt at evaluation time. The intended behavior must therefore be carried by the adapter rather than injected in the prompt.

2.2 Evaluation

We report greedy-decoding pass@1, the fraction of tasks solved by the model's first generated program, on HumanEval (164 problems) [2], HumanEval+ (the EvalPlus augmentation

of HumanEval) [5], and the 500-problem MBPP test split [1]. These benchmarks are separate from the RLVR training sources. HumanEval+ reuses the 164 HumanEval prompts with substantially expanded tests; it is a stricter test suite, not an independent third problem set. All three reported test suites are evaluated at multiple checkpoints, so they function as development benchmarks for checkpoint inspection rather than untouched final tests.

Persona evaluation uses 200 held-out prompts. Style cosine is a project-specific measure, not a standard benchmark. We concatenate a model's 200 replies and calculate a 117-value profile: rates for 101 function words and 11 punctuation marks, plus sentence length, sentence-length variation, word length, vocabulary diversity, and capitalization. We then take cosine similarity with the same profile from the 1,000 SFT target replies. Higher values mean that aggregate surface statistics are more similar; they do not by themselves establish semantic quality or fidelity to the novel. We also report mean per-response distance from the SFT target profile.

Two separate AI judges serve different roles. GPT-5.4-nano is used only during RLAIF: it sees the prompt, target answer, and sampled response and returns task and style rewards. The final GPT-5.4 evaluator removes code and rates each of 200 replies from 1 to 5 on sentence structure, vocabulary, markers, consistency, and human plausibility. It is not the judge used for training.

Table 1 reports the training-trajectory evaluation. A separate final-model rerun appears in Table 3; the two snapshots are reported separately rather than averaged. Each training stage was run once, so the results describe the observed checkpoints rather than variation across repeated training runs.

3 SFT Installs Style and Damages Coding

The SFT set is built by rewriting 1,200 human-written conversations from No Robots, Dolly-15k, and OASST2; 1,000 enter the SFT dataset, 950 are used for optimization after the internal split, and 200 separate rows are used for persona evaluation. This construction holds the question and intended content approximately fixed while changing the response style. It does not hold answer quality exactly fixed because the rewrite model can alter content.

SFT raises style cosine from 0.583 to 0.879, but HumanEval falls from 0.762 to 0.555 and HumanEval+ from 0.738 to 0.506. The data composition offers a plausible mechanism: only 21 of the 1,000 SFT rows use a fenced code block, and the rewritten answers are terse. In one evaluation artifact all 164 base completions contain a code block while none of the 164 SFT completions do. That observation supports a formatting-loss hypothesis, but we did not label every failure as formatting or program logic. We therefore avoid attributing the entire coding decline to formatting.

An earlier synchronized ablation is important context. SFT with self-generated code targets reaches 0.701 on HumanEval and 0.689 on HumanEval+, with an aggregate persona-judge score of 19.725. In that evaluation snapshot it outperforms the earlier multi-objective RLVR checkpoint (0.622 and 0.573; judge 19.271). Those values are not directly comparable to the final RLVR run because the evaluation snapshots differ, but they show that a data intervention is a serious alternative to adding RL stages.

4 RLAIF and a Reward-Gaming Control

4.1 Failed first attempt

The first RLAIF run uses a single judge reward and a prompt that explicitly names catchphrases as desirable. Its held-out reward rises while coding collapses; by the end, 91% of persona replies open with "Listen. Listen." This is a useful control because the outputs visibly optimize a recognizable proxy instead of the intended natural voice. The absolute size of the code collapse is uncertain: the historical evaluator extracted only the first code block, while the model often split code across blocks.

4.2 Revised objective

The revised run separates four signals (Table 2) and increases the KL coefficient from the failed configuration. The decomposition makes it possible to see which proxy is changing rather than hiding all behavior in one scalar reward.

RLAIF suppresses the most conspicuous learned tics but does not clearly improve overall character fidelity. Style cosine rises from 0.879 to 0.930, while the blind canon audit rates RLAIF and SFT similarly. The clearest change is that forced-catchphrase flags fall from 42% to 0%. The penalty also fires on 58% of synthetic target-style references, showing that it is stricter than the distribution it is meant to preserve.

5 Verifiable-Reward RL

5.1 Earlier MBPP-based ablations

The earlier `rlvr` branch contains verifier-only and multi-objective experiments trained on MBPP-derived problems. They test whether executable reward helps and whether persona rewards should remain active during RLVR. However, the training prompt exposes the same asserts used by the verifier, and MBPP is also an evaluation benchmark. Those runs are informative ablations but weak evidence of held-out generalization. We preserve their code and results while excluding them from the headline causal claim.

5.2 Final RLVR run

The final RLVR dataset is ordered by estimated difficulty rather than shuffled. Training begins from the RLAIF adapter and excludes the three reported test suites. All 700 selected IDs are unique and map to matching prompts and tests in the 1,700-problem pool. The source mix is 562 KodCode, 73 Skywork,

Table 1: Training-trajectory checkpoint results. Coding entries are pass@1. "Style cosine" is similarity to the project target corpus.

Checkpoint	HumanEval	HumanEval+	MBPP	Style cosine
Base Qwen	**0.762**	**0.738**	**0.588**	0.583
SFT	0.555	0.506	0.556	0.879
RLAIF	0.579	0.537	0.562	0.930
RLVR, step 174	0.695	0.646	0.584	**0.947**

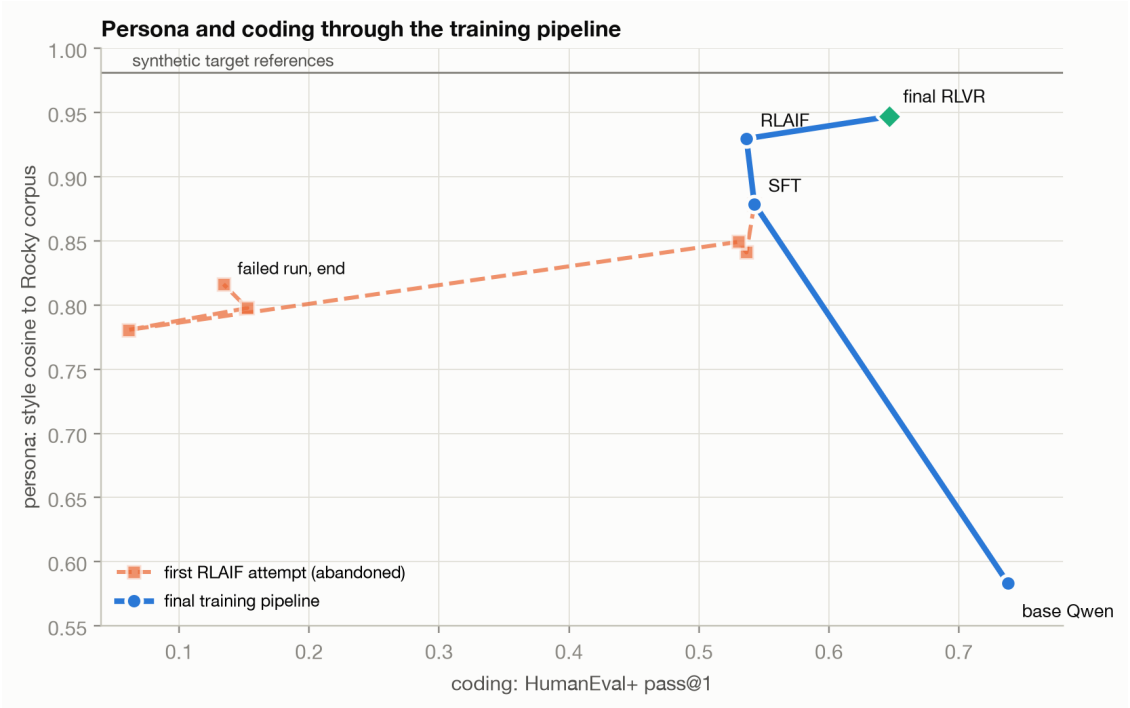


Figure 1: Coding and persona style through the main training path, with the abandoned first RLAIF run retained as a failure control. The apparent improvement in both axes at the final RLVR checkpoint is subject to the metric and replication limitations in Sections 6 and 7.

Table 2: Reward terms in the revised RLAIF run.

Term	Measurement	Weight
Task	AI judge: answer correctness	1.0
Style	AI judge: target voice	0.6
Catchphrase	Rule-based excess-frequency penalty	1.0
Format	One code block; response-length guard	1.0

Table 3: Two evaluations of the final RLVR checkpoint. A dash means the benchmark was not included. Values are reported separately, not averaged.

Evaluation	HumanEval	HumanEval+	MBPP	LCB
Training trajectory	0.695	0.646	0.584	–
Shared final rerun	0.683	0.628	0.584	0.149

34 Klear, and 31 DeepCoder problems. Difficulty is estimated from eight sampled RLAIF completions per problem. The final set contains 113 easy, 201 medium, 316 hard, and 70 very-hard problems, concentrating training on tasks that are neither trivial nor uniformly uninformative.

The binary reward requires exactly one fenced Python block and all visible tests to pass. It supplies no persona or AI-judge reward. In the recorded evaluations, RLVR moves from 95/164 to 114/164 passing HumanEval completions, 88/164 to 106/164 on HumanEval+, and 281/500 to 292/500 on MBPP relative to RLAIF (Figure 5). The last checkpoint remains below the recorded base by 11 HumanEval problems, 15 HumanEval+ tests, and two MBPP problems. These comparisons use one training run and mix evaluation launches; they are development-set evidence, not an unbiased estimate of final generalization.

The shared final-model rerun produces slightly different HumanEval and HumanEval+ generations from the training-trajectory evaluation (Table 3). Greedy vLLM generation was not bitwise repeatable in our systems check, so we report both snapshots rather than choosing the larger one. The matching MBPP result and separate rule-based verifier provide a partial consistency check.

Qualitative inspection of disagreements with the base model suggests many residual errors are semantic near misses: short programs solve a simpler interpretation of the prompt. This is a hypothesis generated from examples, not a coded error analysis. A blinded taxonomy with inter-rater agreement would be

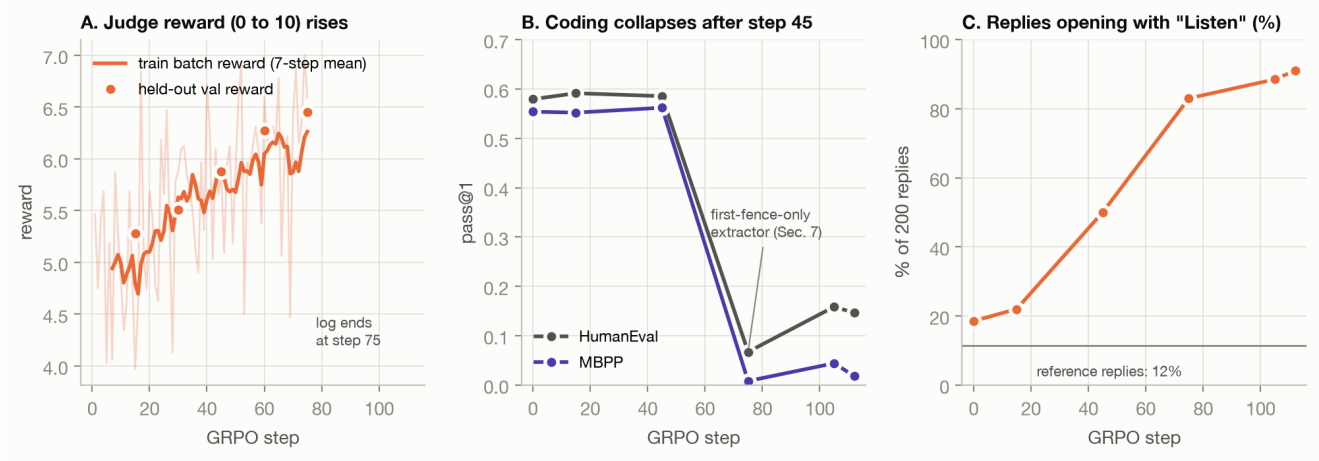


Figure 2: The abandoned first RLAIF run. Judge reward rises, the historical coding measurement collapses after step 45, and one rewarded opener saturates the outputs. The coding curve is partially confounded by the old first-block extractor.

needed to make it a result.

6 Does the Persona Survive?

Across RLVR, style cosine rises from 0.930 to 0.947, while mean per-response centroid distance rises from 1.147 to 1.208. The first metric compares one aggregate profile for the full corpus; the second asks whether each reply resembles a typical target reply. Together they show that the recognizable surface style remains, but not that every reply becomes closer to the synthetic target.

We also conducted two blind checks on earlier checkpoints. One teammate rated 16 replies. Two Claude Opus 5.5 rater instances scored 160 shuffled replies: 40 prompts from each of four sources, evaluated against a canon profile. Mean 1-to-5 fidelity scores were 2.67 for target rewrites, 2.33 for SFT, 2.27 for the failed RLAIF run, and 2.25 for the revised RLAIF run. These close scores do not distinguish the visibly failed run. A targeted flag does: it marks forced catchphrases in 98% of failed-run replies and none of the revised-run replies. We treat this check as qualitative because it relies mainly on model raters and has a small human sample.

The completed GPT-5.4 rubric evaluation scores all 200 final-model replies: sentence structure 4.675, vocabulary 4.280, markers 2.770, consistency 3.585, and human plausibility 3.190, for 18.5/25 overall. Its markers-minus-plausibility gap is -0.42, which does not show the marker-heavy pattern seen in the failed RLAIF control. The rubric uses the same project style construct that informed the training target, however, so it is corroborating evidence rather than independent human ground truth.

7 Limitations and Threats to Validity

No replicated training runs. Each training stage was run once. We therefore cannot measure how results vary across random seeds or separate method effects from sampling and systems variance. Even greedy vLLM generations were not bitwise repeatable in a repository check.

Checkpoint selection and repeated evaluation. The final RLVR checkpoint is reported after inspecting benchmark trajectories. Calling a point "best" is descriptive; it is not an unbiased held-out selection. A clean confirmation would choose a checkpoint using training reward or a validation set and evaluate the three test suites once.

Metric construction. Style cosine, centroid distance, catchphrase flags, and AI rubrics were all designed by the project team. They are neither independent nor externally validated, and some target the same markers used during training. The completed rubric judge is also an AI system, not human ground truth.

Changing more than one variable. The failed and revised RLAIF runs differ in prompt, reward decomposition, learning rate, and KL coefficient. Their contrast diagnoses a bad configuration but does not isolate which change fixes it.

8 Conclusion

The experiment begins with a common post-training failure: SFT successfully installs a surface persona while degrading a capability absent from most demonstrations. A naive AI-judge objective then creates an obvious proxy exploit. Reward decomposition removes that specific observed pattern, and RLVR recovers much of the measured coding loss. The result does not establish that persona and coding are generally compatible, nor that the latest checkpoint is preferable on every persona measure. It does show why failure controls, disaggregated rewards, auditable data, a separate final test, and multiple style metrics are necessary for making a credible claim.

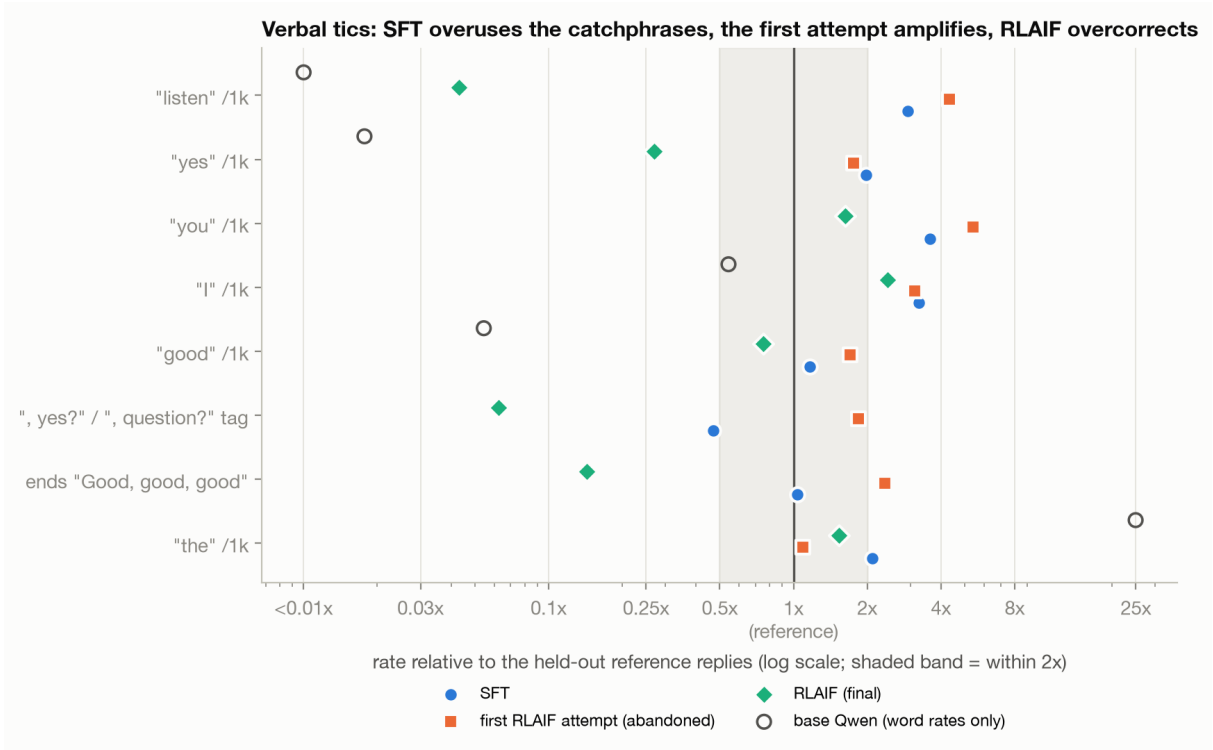


Figure 3: Rates of selected verbal markers relative to held-out target rewrites. The revised RLAIF run suppresses the targeted catchphrases, including some that occur in the target itself. The figure measures feature frequency, not human-likeness or canonical character fidelity.

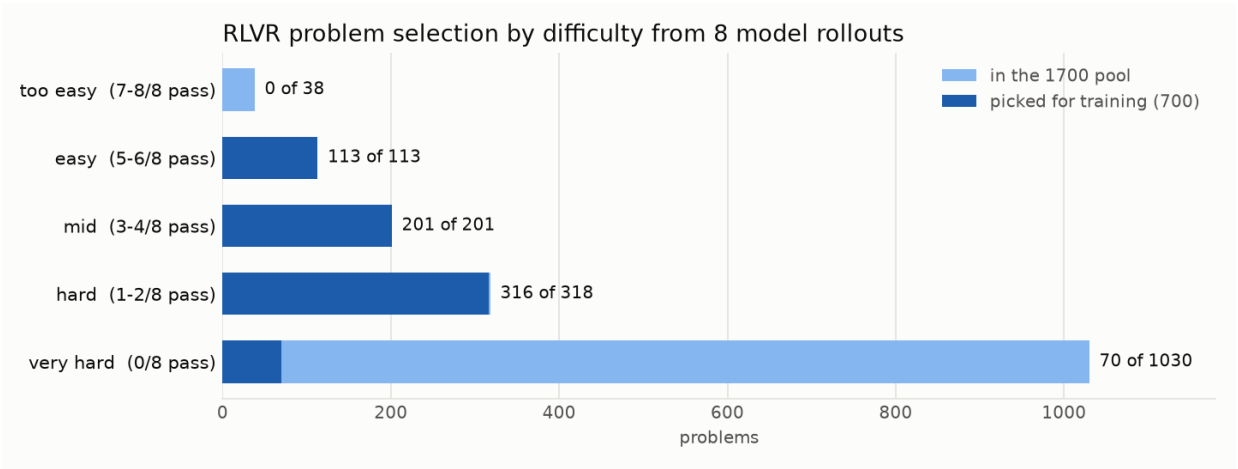


Figure 4: Construction of the 700-problem RLVR dataset from a 1,700-problem pool. Difficulty is the RLAIF model's success count across eight sampled answers. Dark bars indicate the selected training problems.

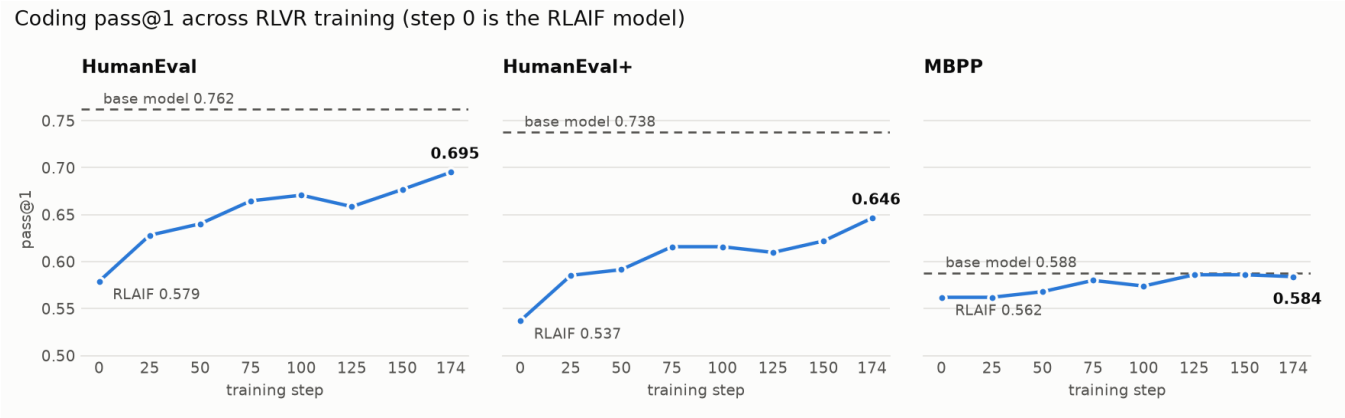


Figure 5: Coding pass@1 during RLVR. Step 0 is the RLAIF model. The final checkpoint is the best observed checkpoint on HumanEval and HumanEval+; MBPP peaks at 0.586 at steps 125 and 150, versus 0.584 at step 174. Repeated inspection makes all checkpoint comparisons exploratory.

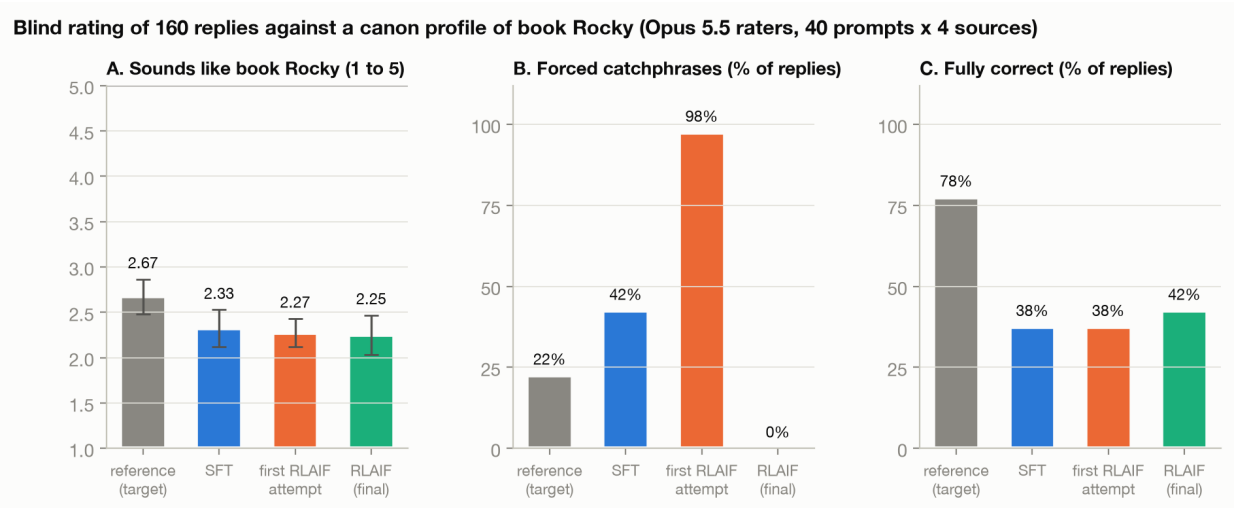


Figure 6: Blind canon-grounded evaluation of 160 replies from four earlier sources. The holistic 1-to-5 score does not separate the visibly gamed first RLAIF run from SFT; the targeted forced-catchphrase flag does.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[4] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, et al. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

[5] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, 2023.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[7] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.